

Software Instalado En La Plataforma Del Cliente

GA10-220501097-AA3-EV01

**Simulación del despliegue local del sistema AutoReserve mediante Docker sobre
Ubuntu Server**

Proyecto: AutoReserve

**Aplicación web para la gestión integral de empresas dedicadas al alquiler de
vehículos.**

Aprendiz:

Hernán Leonardo Quiroga Beltrán

Ficha:

3118323

Instructor:

Jairo Antonio Muñoz

Servicio Nacional de Aprendizaje – SENA

Tecnología en Análisis y Desarrollo de Software

2026

TABLA DE CONTENIDO

- 1. Portada**
- 2. Introducción**
- 3. Objetivos**
- 4. Descripción del proyecto AutoReserve**
- 5. Selección de la plataforma de desarrollo e implantación**
- 6. Preparación del entorno de virtualización**
- 7. Instalación y configuración del entorno de despliegue**
- 8. Generación de los artefactos del sistema AutoReserve**
- 9. Preparación del entorno de despliegue y transferencia del producto**
- 10. Construcción y despliegue mediante Docker**
- 11. Resultados obtenidos**
- 12. Conclusiones**
- 13. Referencias**

Índice de figuras

- Figura 1. Descarga de Oracle VirtualBox.
- Figura 2. Descarga de Ubuntu Server.
- Figura 3. Creación de la máquina virtual en VirtualBox.
- Figura 4. Configuración de memoria RAM y procesador.
- Figura 5. Configuración del disco duro virtual.
- Figura 6. Selección de la imagen ISO de Ubuntu Server.
- Figura 7. Proceso de instalación de Ubuntu Server.
- Figura 8. Finalización de la instalación de Ubuntu Server.
- Figura 9. Inicio de sesión en Ubuntu Server.
- Figura 10. Actualización del sistema operativo.
- Figura 11. Instalación de Docker.
- Figura 12. Instalación de Docker Compose.
- Figura 13. Verificación de versiones de Docker y Docker Compose.
- Figura 14. Generación del build del frontend.
- Figura 15. Resultado de la compilación del frontend.
- Figura 16. Ejecución del Maven Wrapper para generar el backend.
- Figura 17. Corrección de errores durante el proceso de pruebas.
- Figura 18. Generación exitosa del archivo JAR del backend.
- Figura 19. Configuración de perfiles de Spring Boot.
- Figura 20. Configuración y habilitación de OpenSSH.
- Figura 21. Transferencia del archivo JAR mediante SCP.
- Figura 22. Verificación del archivo JAR en Ubuntu Server.
- Figura 23. Transferencia de los archivos del frontend compilado.
- Figura 24. Verificación de los archivos del frontend en Ubuntu Server.
- Figura 25. Creación del Dockerfile del backend.
- Figura 26. Construcción de la imagen Docker del backend.
- Figura 27. Verificación de las imágenes Docker del backend.
- Figura 28. Creación del Dockerfile del frontend.
- Figura 29. Construcción de la imagen Docker del frontend.
- Figura 30. Verificación de las imágenes Docker del frontend.
- Figura 31. Creación del archivo docker-compose.yml.
- Figura 32. Verificación del contenido del archivo docker-compose.yml.
- Figura 33. Ejecución del comando docker compose up -d.
- Figura 34. Inicio exitoso de los tres contenedores.
- Figura 35. Verificación de contenedores mediante docker ps.
- Figura 36. Validación del frontend utilizando curl http://localhost.
- Figura 37. Validación del backend utilizando curl http://localhost:8080.
- Figura 38. Revisión de logs del backend mediante docker logs.
- Figura 39. Evidencia de la creación automática de datos iniciales.

2. Introducción

El despliegue de aplicaciones constituye una de las etapas más importantes dentro del ciclo de vida del desarrollo de software, ya que representa el momento en el cual una solución tecnológica es preparada e instalada para ser utilizada por el cliente final.

Tradicionalmente, este tipo de actividades académicas suelen realizarse mediante la instalación de plantillas web sobre plataformas como XAMPP, WAMP o LAMP. Sin embargo, considerando que AutoReserve corresponde a un proyecto funcional desarrollado durante el programa de formación, se decidió utilizar el producto real como objeto de estudio para simular un proceso de implantación cercano a un escenario empresarial.

Para ello, se empleó una infraestructura basada en Ubuntu Server 24.04 LTS ejecutándose sobre Oracle VirtualBox, complementada con Docker y Docker Compose como plataforma de despliegue. Este enfoque permite reproducir las condiciones que enfrentaría una organización al adquirir el software, facilitando la instalación, actualización y mantenimiento de sus componentes.

El presente informe documenta paso a paso la preparación del entorno, la construcción del producto y la simulación de instalación local de AutoReserve, incorporando evidencias fotográficas obtenidas durante la ejecución práctica del procedimiento.

3. Objetivos

3.1 Objetivo general

Realizar la simulación de instalación local del sistema AutoReserve sobre una plataforma de despliegue basada en contenedores, documentando el procedimiento ejecutado mediante evidencias fotográficas y descripciones técnicas.

3.2 Objetivos específicos

- Seleccionar una plataforma adecuada para el despliegue del sistema.
- Preparar el entorno del cliente mediante Ubuntu Server, Docker y Docker Compose.
- Construir los componentes necesarios para la instalación del aplicativo.
- Simular el proceso de despliegue que experimentaría un cliente al adquirir AutoReserve.
- Validar el correcto funcionamiento del sistema instalado.
- Documentar el procedimiento realizado mediante capturas de pantalla integradas al informe.

4. Descripción del producto a instalar

AutoReserve es una aplicación web desarrollada para optimizar la gestión operativa y administrativa de empresas dedicadas al alquiler de vehículos. El sistema permite centralizar procesos relacionados con reservas, administración de automóviles, gestión de usuarios y atención de solicitudes mediante una interfaz accesible desde cualquier navegador web.

El proyecto se encuentra actualmente en una fase avanzada de desarrollo correspondiente a un Producto Mínimo Viable (MVP), habiendo superado satisfactoriamente sus pruebas funcionales.

Tecnologías utilizadas

Componente	Tecnología
Frontend	React + Vite
Backend	Java 21 + Spring Boot
Seguridad	Spring Security + JWT
Base de datos	MySQL
Gestión de dependencias	Maven
Plataforma de despliegue	Docker
Orquestación	Docker Compose

Funcionalidades principales

- Registro e inicio de sesión.
- Gestión de usuarios.
- Gestión de vehículos.
- Gestión de categorías.
- Gestión de sedes.
- Gestión de reservas.
- Gestión de tickets.
- Panel administrativo.
- Consulta de disponibilidad.
- Sistema de favoritos.

5. Plataforma seleccionada para el despliegue

Aunque el componente formativo presenta alternativas tradicionales como LAMP, XAMPP o WAMP, para la presente evidencia se seleccionó una arquitectura basada en Docker debido a sus ventajas en términos de portabilidad, facilidad de instalación y replicabilidad.

La solución implementada está conformada por los siguientes elementos:

- Oracle VirtualBox como plataforma de virtualización.
- Ubuntu Server 24.04 LTS como sistema operativo del servidor.
- Docker Engine para la ejecución de contenedores.
- Docker Compose para la orquestación del despliegue.

Esta combinación tecnológica representa una práctica ampliamente utilizada en entornos modernos de desarrollo e implantación de software.

Evidencias que debes insertar aquí

Figura 1.

Descarga de Ubuntu Server.

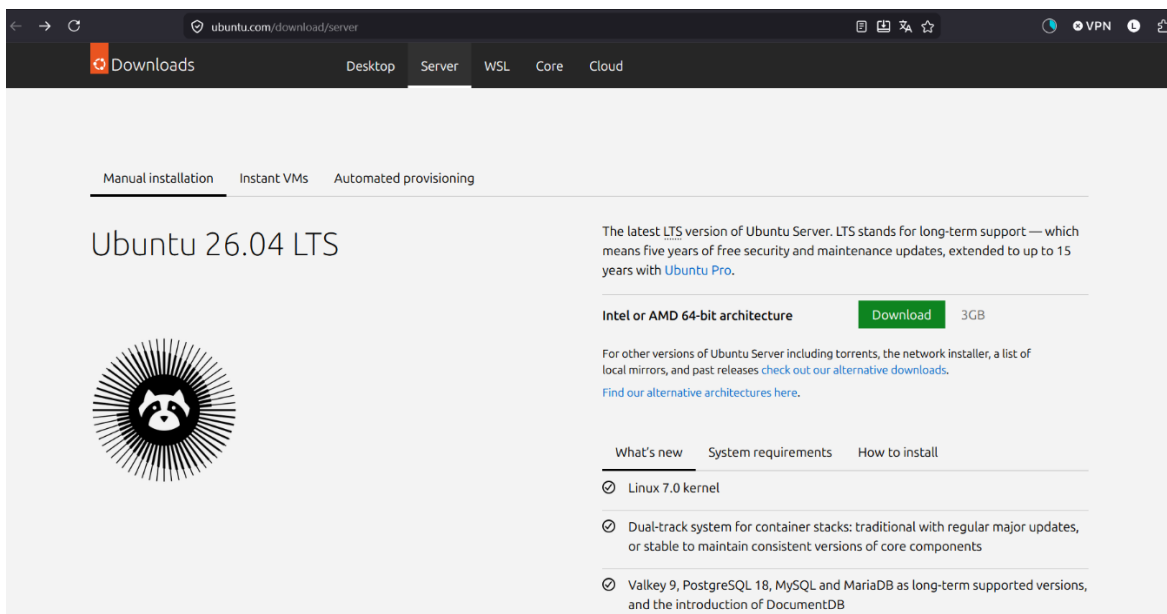


Figura 2.

Descarga de Oracle VirtualBox.

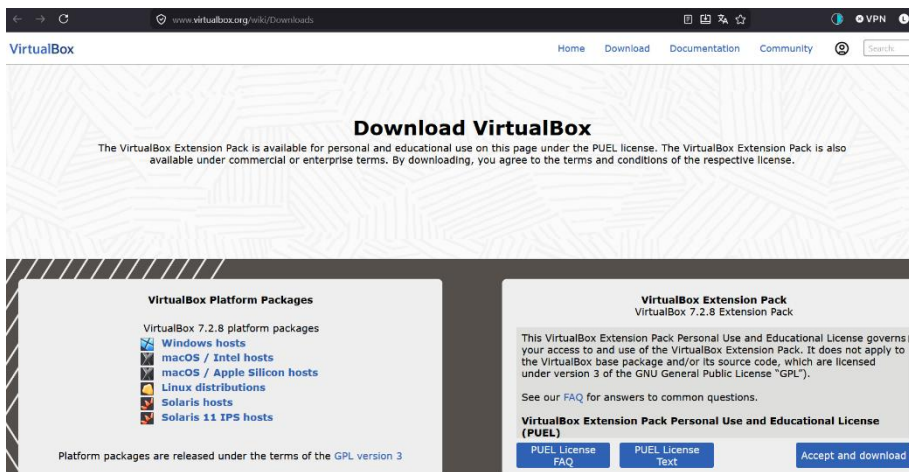


Figura 3.

Verificación de Docker instalado.

docker --versión

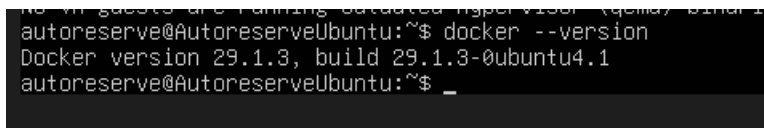
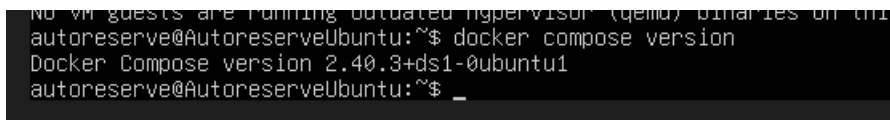


Figura 4.

Verificación de Docker Compose.

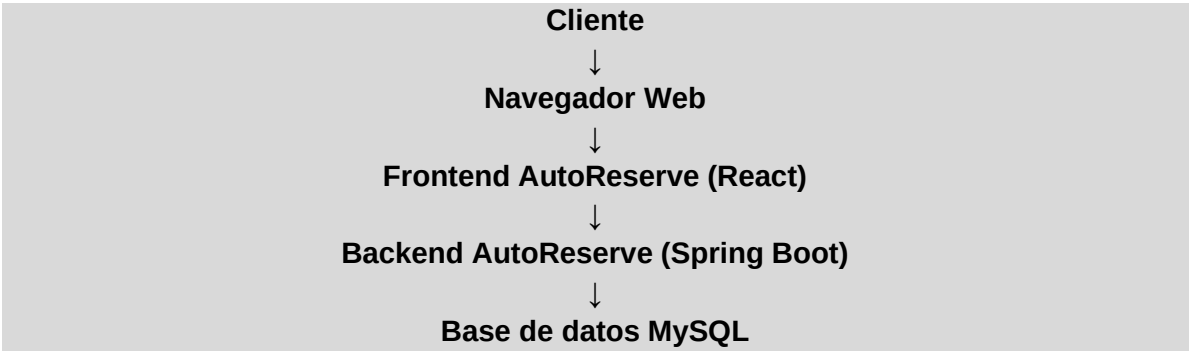
docker compose versión



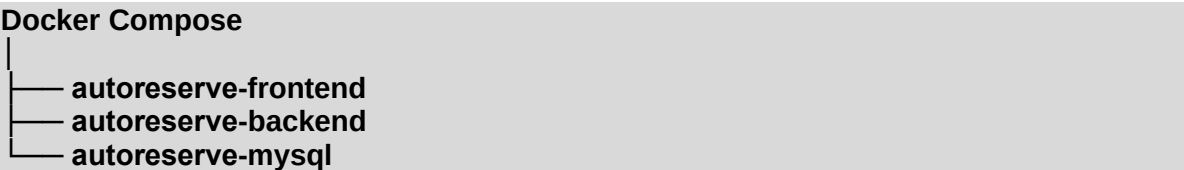
6. Arquitectura propuesta de despliegue

La instalación simulada de AutoReserve se realizará mediante una arquitectura multicontenedor, separando cada uno de los componentes del sistema.

La distribución propuesta es la siguiente:



Cada componente será desplegado dentro de un contenedor independiente administrado por Docker Compose.



7. Prerrequisitos del cliente

Antes de proceder con la instalación de AutoReserve, se verificó que la infraestructura del cliente cumpliera con las características mínimas requeridas.

Requisitos mínimos

Recurso	Requerimiento
Sistema operativo	Ubuntu Server 24.04 LTS
Procesador	2 vCPU
Memoria RAM	8 GB
Almacenamiento	100 GB SSD
Docker	Instalado
Docker Compose	Instalado
Conectividad	Acceso a Internet

Figura 5.

Validación del procesador.

Iscpu

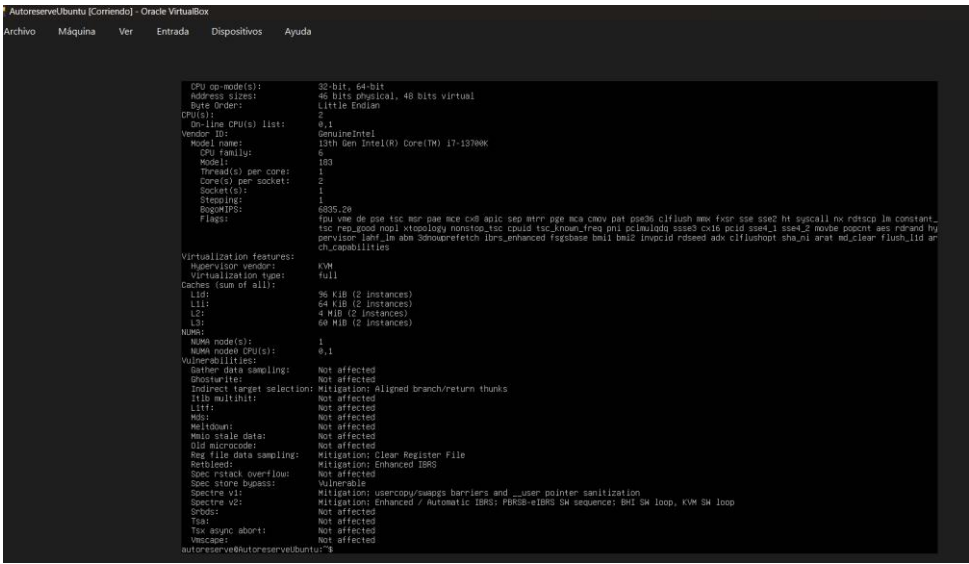


Figura 6.

Validación de memoria RAM.

free -h

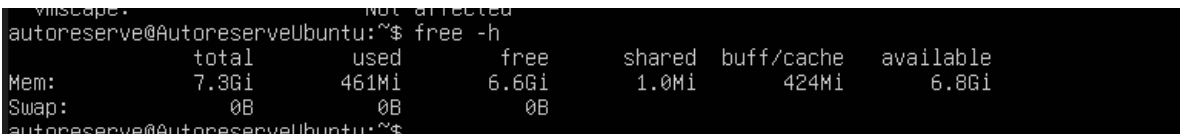


Figura 7.

Validación del almacenamiento.

df -h

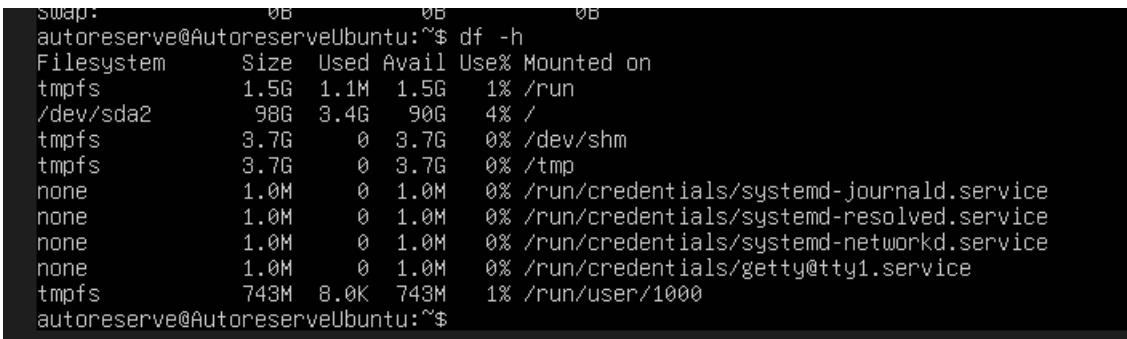


Figura 8.

Validación del sistema operativo.

lsb_release -a

```
autoreserve@AutoreserveUbuntu:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:   Ubuntu 26.04 LTS
Release:      26.04
Codename:     noble
autoreserve@AutoreserveUbuntu:~$
```

8. Preparación del producto para el despliegue

Una vez validado el entorno del cliente, se procedió a preparar los componentes que conforman AutoReserve para su futura ejecución mediante contenedores Docker.

Debido a que la aplicación está compuesta por un frontend desarrollado en React y Vite, un backend construido con Java y Spring Boot, y una base de datos MySQL inicializada automáticamente por el propio sistema, fue necesario generar los artefactos ejecutables correspondientes a cada componente.

8.1 Preparación del frontend

El frontend de AutoReserve fue compilado con el objetivo de generar una versión optimizada para producción.

Para ello, se ejecutó el comando correspondiente desde la carpeta raíz del proyecto frontend.

Comando ejecutado:

```
npm run build
```

Este proceso genera los archivos estáticos que posteriormente serán incluidos dentro de un contenedor Docker encargado de servir la interfaz web del sistema.

Figura 9. Construcción del frontend mediante npm run build.

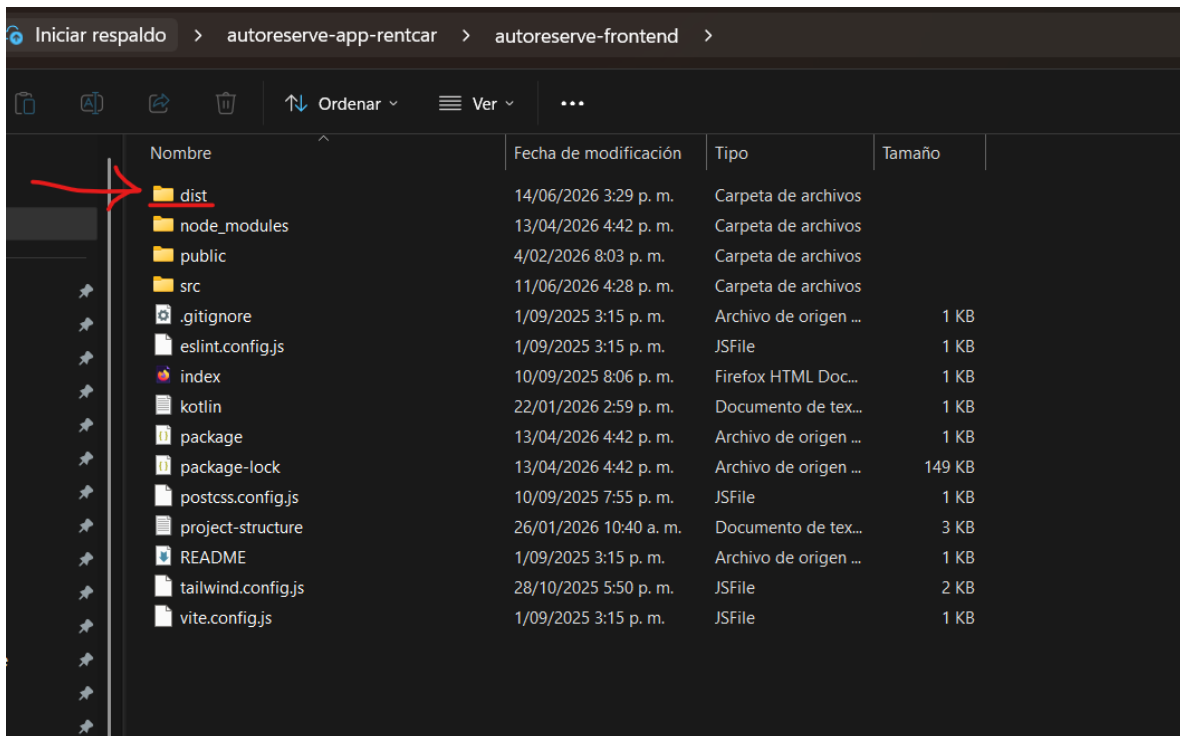
Figura 10. Resultado de la construcción del frontend.

```
(base) PS C:\Users\NatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-frontend> npm run build

> autoreserve@0.0.0 build
> vite build

vite v7.1.5 building for production...
✓ 116 modules transformed.
[plugin vite:reporter]
(!) C:\Users\NatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-frontend\src\api\http.js is dynamically imported by C:\Users\NatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-frontend\src\pages\AdminAutos.jsx but also statically imported by C:\Users\NatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-frontend\src\api\adminBranchesApi.js, C:\Users\NatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-frontend\src\api\adminCarsApi.js, C:\Users\NatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-frontend\src\api\adminCategoriesApi.js, C:\Users\NatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-frontend\src\api\adminReservationsApi.js, C:\Users\NatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-frontend\src\api\adminStatsApi.js, C:\Users\NatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-frontend\src\api\adminUsersApi.js, C:\Users\NatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-frontend\src\api\branchesApi.js, C:\Users\NatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-frontend\src\api\carsApi.js, C:\Users\NatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-frontend\src\api\categoriesApi.js, C:\Users\NatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-frontend\src\api\contactApi.js, C:\Users\NatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-frontend\src\api\faqApi.js, C:\Users\NatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-frontend\src\api\favoritesApi.js, C:\Users\NatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-frontend\src\api\policiesApi.js, C:\Users\NatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-frontend\src\api\reservationsApi.js, C:\Users\NatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-frontend\src\api\searchApi.js, C:\Users\NatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-frontend\src\api\userApi.js, C:\Users\NatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-frontend\src\auth\authApi.js, dynamic import will not move module into another chunk.

dist/index.html      0.48 kB | gzip: 0.30 kB
dist/assets/logo-Ckn8-0pq.png 44.88 kB
dist/assets/index-Be3RIL63.css 33.31 kB | gzip: 6.12 kB
dist/assets/index-BrpsCws_.js 442.10 kB | gzip: 115.33 kB
✓ built in 1.01s
(base) PS C:\Users\NatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-frontend>
```



8.2 Preparación del backend

Posteriormente, se procedió a construir el backend desarrollado con Spring Boot utilizando Maven.

Comando ejecutado:

```
.!mvnw.cmd clean package
```

El objetivo de este procedimiento consiste en generar un archivo ejecutable que contenga toda la lógica de negocio de AutoReserve y pueda ser desplegado dentro de un contenedor independiente.

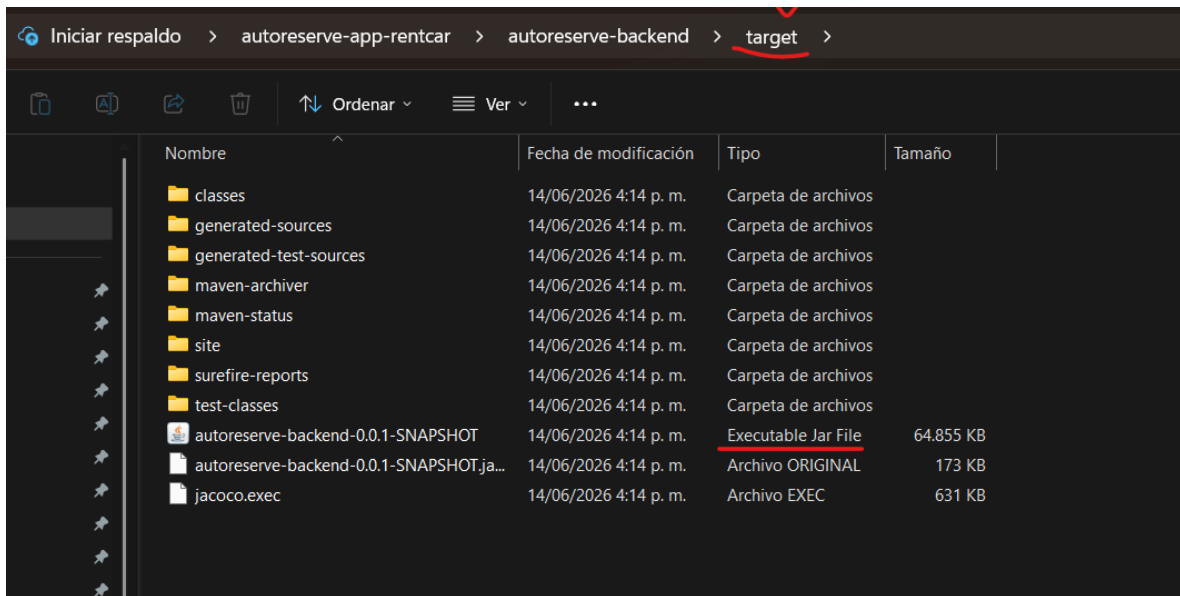
Figura 11. Construcción del backend mediante Maven.

Figura 12. Resultado de la construcción del backend.

```

Administrator: C:\Program Files\PowerShell\powershell.exe
Hibernate: select r1.0.id,r1.0.name from role r1.0 where r1.0.name=?
Hibernate: select u1.0.id,u1.0.created_at,u1.0.first_name,u1.0.last_name,u1.0.password_hash,u1.0.phone,u1.0.role_id,u1.0.status from 'user' u1.0 where u1.0.email=?
Hibernate: select r1.0.id,r1.0.name from role r1.0 where r1.0.id=?
Hibernate: select u1.0.id,u1.0.created_at,u1.0.first_name,u1.0.last_name,u1.0.password_hash,u1.0.phone,u1.0.role_id,u1.0.status from 'user' u1.0 where u1.0.email=?
Hibernate: select r1.0.id,r1.0.name from role r1.0 where r1.0.id=?
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.776 s -- in com.autoreserve.backend.web.controller.CategoryControllerTest
[INFO] Running com.autoreserve.backend.web.controller.SearchControllerTest
2026-06-14T16:14:45.710-05:00 INFO 26448 --- [autoreserve-backend] [main] t.c.s.AnnotationConfigContextLoaderUtils : Could not detect default configuration classes for test class [com.autoreserve.bac
kend.web.controller.SearchControllerTest]: SearchControllerTest does not declare any static, non-private, non-final, nested classes annotated with @Configuration.
2026-06-14T16:14:45.712-05:00 INFO 26448 --- [autoreserve-backend] [main] .b.t.c.SpringBootTestContextBootstrapper : Found @SpringBootTestConfiguration com.autoreserve.backend.AutoreserveBackendApplicati
on for test class com.autoreserve.backend.web.controller.SearchControllerTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.018 s -- in com.autoreserve.backend.web.controller.SearchControllerTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 31, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.11:report (report) @ autoreserve-backend ---
[INFO] Loading execution data file C:\Users\MatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-backend\target\jacoco.exec
[INFO] Analyzed bundle 'autoreserve-backend' with 92 classes
[INFO]
[INFO] --- jar:3.4.2:jar (default-jar) @ autoreserve-backend ---
[INFO] Building jar: C:\Users\MatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-backend\target\autoreserve-backend-0.0.1-SNAPSHOT.jar
[INFO]
[INFO] --- spring-boot:3.4.1:repackage (repackage) @ autoreserve-backend ---
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/springframework/boot/spring-boot-loader-tools/3.4.1/spring-boot-loader-tools-3.4.1.jar
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/apache/httpcomponents/core5/httpcore5/5.3.1/httpcore5-5.3.1.jar
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/apache/httpcomponents/core5/httpcore5-h2/5.3.1/httpcore5-h2-5.3.1.jar
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/springframework/boot/spring-boot-buildpack-platform/3.4.1/spring-boot-buildpack-platform-3.4.1.jar
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/apache/httpcomponents/client5/httpclient5/5.4.1/httpclient5-5.4.1.jar
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/httpcomponents/core5/httpcore5-h2/5.3.1/httpcore5-h2-5.3.1.jar (241 kB at 362 kB/s)
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/springframework/boot/spring-boot-buildpack-platform/3.4.1/spring-boot-buildpack-platform-3.4.1.jar (287 kB at 434 kB/s)
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/springframework/boot/spring-boot-loader-tools/3.4.1/spring-boot-loader-tools-3.4.1.jar (465 kB at 639 kB/s)
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/httpcomponents/client5/httpclient5/5.4.1/httpclient5-5.4.1.jar (908 kB at 1.2 MB/s)
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/httpcomponents/core5/httpcore5/5.3.1/httpcore5-5.3.1.jar (906 kB at 1.1 MB/s)
[INFO] Replacing main artifact C:\Users\MatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-backend\target\autoreserve-backend-0.0.1-SNAPSHOT.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to C:\Users\MatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-backend\target\autoreserve-backend-0.0.1-SNAPSHOT.jar.original
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 18.112 s
[INFO] Finished at: 2026-06-14T16:14:48-05:00
[INFO]
(base) PS C:\Users\MatyLeo\Desktop\autoreserve-app-rentcar\autoreserve-backend>

```



La generación satisfactoria de ambos artefactos constituye un requisito indispensable para la posterior construcción de imágenes Docker.

9. Preparación del entorno de despliegue y transferencia del producto

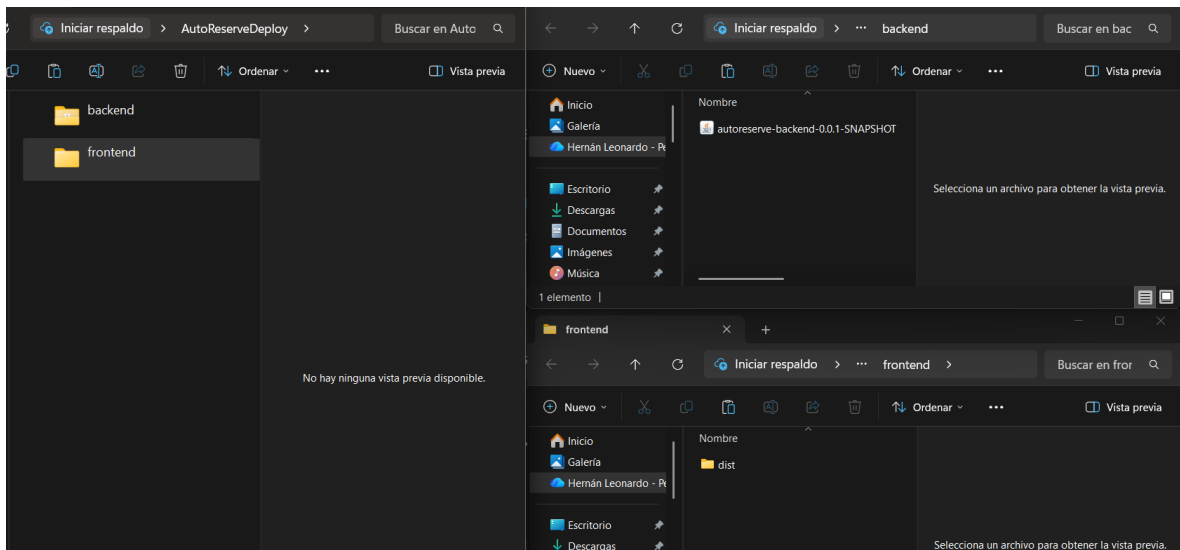
Con el propósito de simular un escenario real de instalación del sistema AutoReserve en la infraestructura tecnológica de un cliente, se llevó a cabo la preparación del servidor destinado al despliegue del aplicativo. Para ello, se utilizó una máquina virtual Ubuntu Server ejecutada sobre Oracle VirtualBox, reproduciendo las condiciones habituales de instalación en un entorno empresarial.

Figura 13. Creación del directorio de despliegue en el servidor.

```
autoreserve@AutoreserveUbuntu:~$ mkdir -p ~/autoreserve-deply
autoreserve@AutoreserveUbuntu:~$ cd ~/autoreserve-deply
autoreserve@AutoreserveUbuntu:~/autoreserve-deply$
```

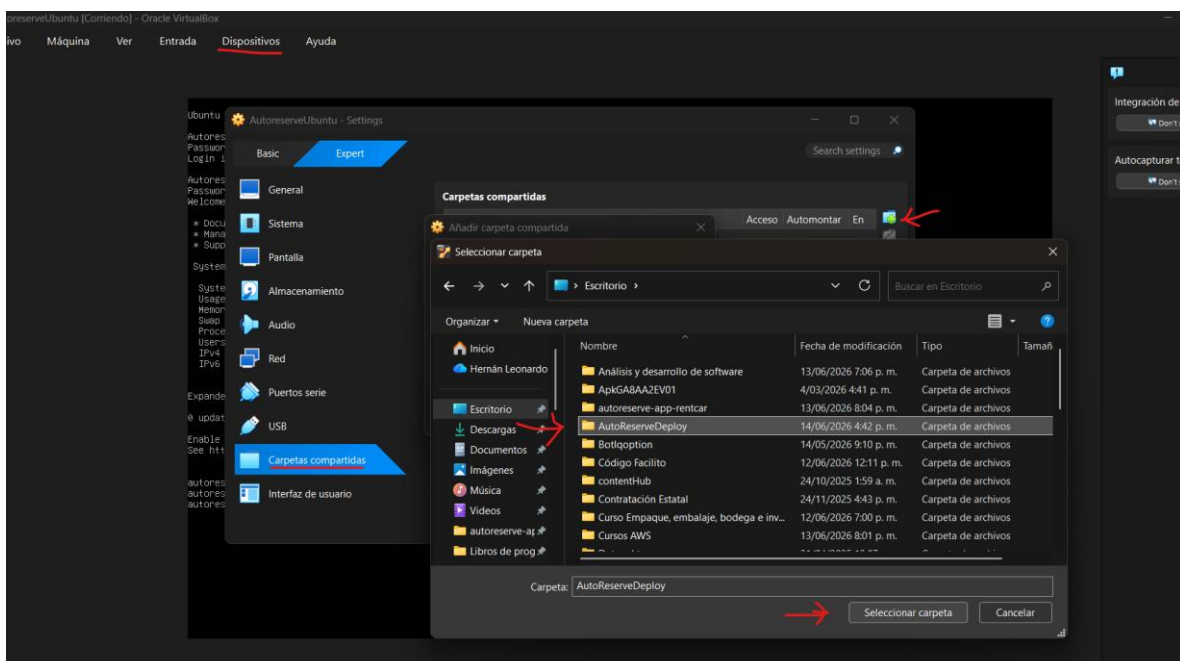
Inicialmente, se organizaron los artefactos generados durante la fase de construcción del proyecto. Estos archivos corresponden al frontend compilado mediante Vite y al backend empaquetado como archivo ejecutable Java (.jar), los cuales constituyen los componentes que posteriormente serán desplegados en el servidor.

Figura 14. Estructura inicial del proyecto AutoReserve y ubicación de los artefactos generados para el despliegue.



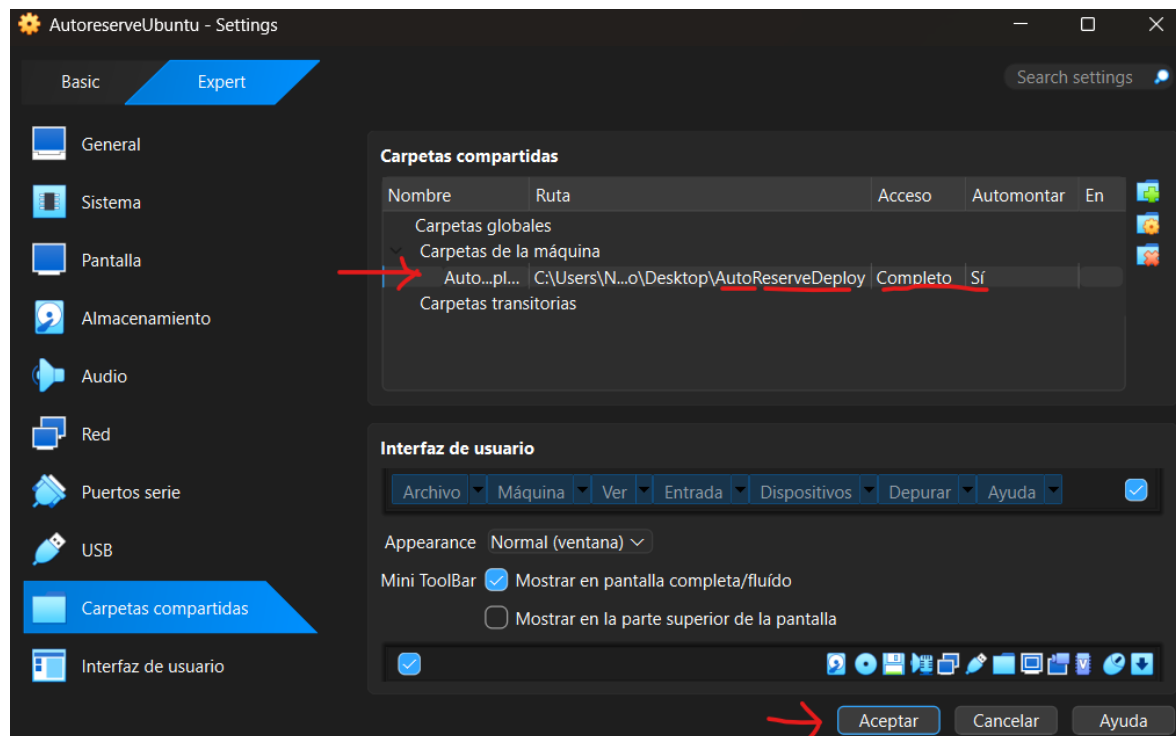
Posteriormente, se intentó implementar un mecanismo de transferencia de archivos mediante la funcionalidad de carpetas compartidas proporcionada por VirtualBox. Para ello, se configuró una carpeta del sistema anfitrión con permisos de acceso completo y montaje automático.

Figura 14. Configuración de carpeta compartida en VirtualBox.



Una vez configurada la carpeta compartida, se verificó que la configuración hubiese quedado almacenada correctamente dentro de la máquina virtual.

Figura 15. Verificación de la configuración de la carpeta compartida en VirtualBox.



Sin embargo, durante las pruebas realizadas desde Ubuntu Server, se evidenció que el acceso automático a la carpeta compartida no estaba disponible debido a la ausencia de los complementos Guest Additions propios de VirtualBox en el sistema operativo invitado. Esta situación llevó a evaluar mecanismos alternativos de transferencia más acordes con entornos profesionales.

Figura 16. Intento de acceso a la carpeta compartida desde Ubuntu Server.

```
autoreserve@AutoreserveUbuntu:~$ ls /media
autoreserve@AutoreserveUbuntu:~$ ls /media/AutoReserveDeploy
ls: cannot access '/media/AutoReserveDeploy': No such file or directory
autoreserve@AutoreserveUbuntu:~$ _
```

Como solución alternativa, se decidió implementar un esquema de administración remota utilizando el protocolo SSH (Secure Shell), ampliamente utilizado para la gestión segura de

servidores Linux. Para ello, se instaló el servicio OpenSSH Server dentro del sistema Ubuntu.

Figura 17. Instalación del servicio OpenSSH Server en Ubuntu Server.

sudo apt install openssh-server -y

```
Get:3 http://archive.ubuntu.com/ubuntu/resolute-updates/main amd64 openssh-sftp-server amd64 1:10.2p1-2ubuntu3.2 [37.5 kB]
Get:4 http://archive.ubuntu.com/ubuntu/resolute-updates/main amd64 openssh-server amd64 1:10.2p1-2ubuntu3.2 [622 kB]
Get:5 http://archive.ubuntu.com/ubuntu/resolute/main amd64 ssh-import-id all 5.11-0ubuntu4 [10.2 kB]
Fetched 998 kB in 0s (2,312 kB/s)
Preconfiguring packages...
Selecting previously unselected package liburp0:amd64.
(Reading database... 96168 files and directories currently installed.)
Preparing to unpack .../liburp0_7.6.q-36build2_amd64.deb...
Unpacking liburp0:amd64 (7.6.q-36build2)...
Selecting previously unselected package ncurses-term.
Preparing to unpack .../ncurses-term_6.6+20251231-1_all.deb...
Unpacking ncurses-term (6.6+20251231-1)...
Selecting previously unselected package openssh-sftp-server.
Preparing to unpack .../openssh-sftp-server_1%3a10.2p1-2ubuntu3.2_amd64.deb...
Unpacking openssh-sftp-server (1:10.2p1-2ubuntu3.2)...
Selecting previously unselected package openssh-server.
Preparing to unpack .../openssh-server_1%3a10.2p1-2ubuntu3.2_amd64.deb...
Unpacking openssh-server (1:10.2p1-2ubuntu3.2)...
Selecting previously unselected package ssh-import-id.
Preparing to unpack .../ssh-import-id_5.11-0ubuntu4_all.deb...
Unpacking ssh-import-id (5.11-0ubuntu4)...
Setting up openssh-sftp-server (1:10.2p1-2ubuntu3.2)...
Setting up ssh-import-id (5.11-0ubuntu4)...
Setting up liburp0:amd64 (7.6.q-36build2)...
Setting up ncurses-term (6.6+20251231-1)...
Setting up openssh-server (1:10.2p1-2ubuntu3.2)...
Creating config file /etc/ssh/sshd_config with new version
Creating user 'sshd' (sshd user) with UID 983 and GID 65534.
Created symlink '/etc/systemd/system/sockets.target.wants/ssh.socket' → '/usr/lib/systemd/system/ssh.socket'.
Created symlink '/etc/systemd/system/ssh.service.requires/ssh.socket' → '/usr/lib/systemd/system/ssh.socket'.
Created symlink '/etc/systemd/system/ssh.service.wants/sshd-keygen.service' → '/usr/lib/systemd/system/sshd-keygen.service'.
Created symlink '/etc/systemd/system/sshd.service.wants/sshd-keygen.service' → '/usr/lib/systemd/system/sshd-keygen.service'.
Created symlink '/etc/systemd/system/sshd@.service.wants/sshd-keygen.service' → '/usr/lib/systemd/system/sshd-keygen.service'.
Created symlink '/etc/systemd/system/ssh.socket.wants/sshd-keygen.service' → '/usr/lib/systemd/system/sshd-keygen.service'.
Processing triggers for ufw (0.36.2-9build1)...
Processing triggers for man-db (2.13.1-1build1)...
Processing triggers for libc-bin (2.43-2ubuntu2)...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
autoreserve@AutoreserveUbuntu:~/autoreserve-deploy$
```

Posteriormente, se verificó que el servicio SSH hubiese quedado correctamente habilitado y en estado de ejecución.

Figura 18. Verificación del estado del servicio SSH.

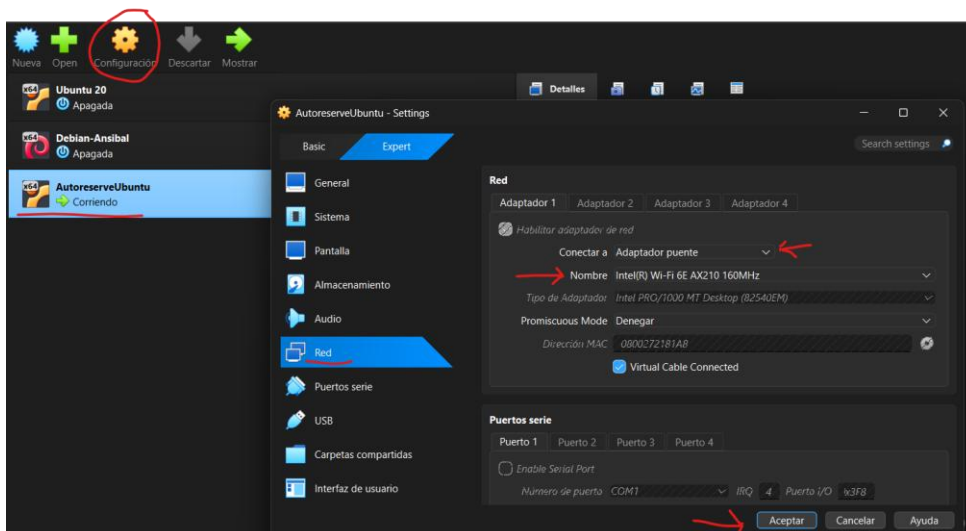
sudo systemctl status ssh

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
autoreserve@AutoreserveUbuntu:~/autoreserve-deploy$ sudo systemctl status ssh
* ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
   Active: inactive (dead)
TriggeredBy: ● ssh.socket
   Docs: man:sshd(8)
        man:sshd_config(5)
autoreserve@AutoreserveUbuntu:~/autoreserve-deploy$
```


Durante la primera prueba de conectividad se identificó que la máquina virtual se encontraba utilizando el modo de red NAT, obteniendo una dirección IP privada no accesible desde el equipo anfitrión.

Con el fin de permitir la administración remota desde el equipo del desarrollador, se modificó la configuración de red de la máquina virtual, reemplazando el modo NAT por un adaptador puente (Bridge Adapter), integrando así el servidor virtual a la misma red local del equipo anfitrión.

Figura 20. Configuración del adaptador de red en modo puente en VirtualBox.



Una vez aplicada la nueva configuración, se obtuvo una nueva dirección IP perteneciente a la red local, habilitando la comunicación directa entre ambos equipos.

Figura 21. Obtención de la nueva dirección IP del servidor Ubuntu.

ip addr

```
autoreserve@AutoreserveUbuntu:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
   link/ether 08:00:27:21:81:a8 brd ff:ff:ff:ff:ff:ff
   altname enx0800272181a8
   inet 192.168.0.12/24 metric 100 brd 192.168.0.255 scope global dynamic enp0s3
       valid_lft 86388sec preferred_lft 86388sec
   inet6 fe80::a00:27ff:fe21:81a8/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
   link/ether 76:36:dc:5d:27:d7 brd ff:ff:ff:ff:ff:ff
   inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
autoreserve@AutoreserveUbuntu:~$
```

Utilizando la nueva dirección IP asignada, se estableció exitosamente una conexión remota mediante SSH desde Windows hacia Ubuntu Server, validando así la capacidad de administración remota del servidor.

Figura 22. Conexión remota mediante SSH desde Windows hacia Ubuntu Server.

```
autoreserve@AutoreserveUbuntu: ~
(base) PS C:\WINDOWS\system32> ssh autoreserve@192.168.0.12
The authenticity of host '192.168.0.12 (192.168.0.12)' can't be established.
ED25519 key fingerprint is SHA256:UMZa1FGqt5dqk85T6qRZaCtFxb0jq+jJ8sQu1XubMVQ.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.0.12' (ED25519) to the list of known hosts.
autoreserve@192.168.0.12's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-22-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun Jun 14 10:35:05 PM UTC 2026

System load:  0.01      Processes:      118
Usage of /:   3.4% of 97.87GB  Users logged in: 0
Memory usage: 4%        IPv4 address for enp0s3: 192.168.0.12
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

autoreserve@AutoreserveUbuntu:~$
```

Adicionalmente, se evidenció la posibilidad de administrar simultáneamente el servidor tanto desde la consola local de VirtualBox como desde la terminal remota del sistema anfitrión.

Figura 23. Administración simultánea del servidor Ubuntu desde VirtualBox y PowerShell.

```
Ubuntu 26.04 LTS AutoreserveUbuntu tty1
AutoreserveUbuntu login: autoreserve
Password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-22-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun Jun 14 10:28:15 PM UTC 2026

System load:  1.5      Processes:      124
Usage of /:   3.4% of 97.87GB  Users logged in: 0
Memory usage: 4%        IPv4 address for enp0s3: 192.168.0.12
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

autoreserve@AutoreserveUbuntu:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast
    link/ether 08:00:27:21:81:a8 brd ff:ff:ff:ff:ff:ff
    altname enx00272181a8
    inet 192.168.0.12/24 metric 100 brd 192.168.0.255 scope global dynamic
        valid_lft 86388sec preferred_lft 86388sec
    inet6 fe80::a00:27ff:fe21:81a8/64 scope link proto kernel llm
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue
    link/ether 76:3c:dc:5d:27:07 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
autoreserve@AutoreserveUbuntu:~$
```

```
autoreserve@AutoreserveUbuntu: ~
(base) PS C:\WINDOWS\system32> ssh autoreserve@192.168.0.12
The authenticity of host '192.168.0.12 (192.168.0.12)' can't be established.
ED25519 key fingerprint is SHA256:UMZa1FGqt5dqk85T6qRZaCtFxb0jq+jJ8sQu1XubMVQ.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.0.12' (ED25519) to the list of known hosts.
autoreserve@192.168.0.12's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-22-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun Jun 14 10:35:05 PM UTC 2026

System load:  0.01      Processes:      118
Usage of /:   3.4% of 97.87GB  Users logged in: 0
Memory usage: 4%        IPv4 address for enp0s3: 192.168.0.12
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

autoreserve@AutoreserveUbuntu:~$
```

Posteriormente, se creó la estructura de directorios destinada a alojar los componentes del despliegue de AutoReserve. Esta organización contempla directorios independientes para el frontend, backend y persistencia de datos, facilitando futuras labores de mantenimiento y actualización.

Figura 24. Creación de la estructura de directorios para el despliegue de AutoReserve.

```
autoreserve@AutoreserveUbuntu:~$ mkdir -p ~/autoreserve-deploy/frontend
autoreserve@AutoreserveUbuntu:~$ mkdir -p ~/autoreserve-deploy/backend
autoreserve@AutoreserveUbuntu:~$ mkdir -p ~/autoreserve-deploy/mysql
autoreserve@AutoreserveUbuntu:~$ tree ~/autoreserve-deploy
Command 'tree' not found, but can be installed with:
sudo snap install tree # version 2.1.3+pkg-5852, or
sudo apt install tree # version 2.3.1-1
See 'snap info tree' for additional versions.
autoreserve@AutoreserveUbuntu:~$ sudo apt install tree -y
[sudo: authenticate] Password:
Installing:
  tree

Summary:
Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 0
Download size: 53.5 kB
Space needed: 125 kB / 96.2 GB available

Get:1 http://archive.ubuntu.com/ubuntu resolute/universe amd64 tree amd64 2.3.1-1 [53.5 kB]
Fetched 53.5 kB in 3s (17.5 kB/s)
Selecting previously unselected package tree.
(Reading database... 99123 files and directories currently installed.)
Preparing to unpack ../tree_2.3.1-1_amd64.deb...
Unpacking tree (2.3.1-1)...
Setting up tree (2.3.1-1)...
Processing triggers for man-db (2.13.1-1build1)...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
autoreserve@AutoreserveUbuntu:~$ tree ~/autoreserve-deploy
/home/autoreserve/autoreserve-deploy
├── backend
├── frontend
└── mysql

4 directories, 0 files
autoreserve@AutoreserveUbuntu:~$
```

Una vez preparada la estructura del servidor, se procedió a transferir el archivo ejecutable del backend utilizando el protocolo SCP (Secure Copy Protocol), garantizando una transmisión segura del artefacto generado durante el proceso de construcción del aplicativo.

Figura 25. Transferencia del archivo ejecutable del backend mediante SCP.

```
scp "C:\Users\NatyLeo\Desktop\AutoReserveDeploy\backend\autoreserve-backend-0.0.1-SNAPSHOT.jar" autoreserve@192.168.0.12:/home/autoreserve/autoreserve-
```

deploy/backend/



```
Administrador: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

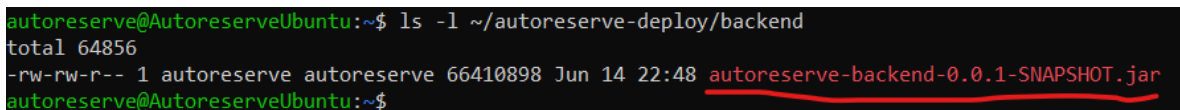
Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

Cargar los perfiles personales y de sistema tardó 1414ms.
(base) PS C:\WINDOWS\system32> scp "C:\Users\NatyLeo\Desktop\AutoReserveDeploy\backend\autoreserve-backend-0.0.1-SNAPSHOT.jar" autoreserve@192.168.0.12:/home/autoreserve/autoreserve-deploy/backend/
autoreserve@192.168.0.12's password:
autoreserve-backend-0.0.1-SNAPSHOT.jar                                100% 63MB 26.6MB/s 00:02
(base) PS C:\WINDOWS\system32>
```

Finalmente, se verificó la presencia del archivo transferido dentro del directorio correspondiente del servidor Ubuntu, confirmando que el componente backend quedó disponible para la siguiente fase del despliegue basada en contenedores Docker.

Figura 26. Verificación del archivo ejecutable del backend en el servidor Ubuntu.

ls -l ~/autoreserve-deploy/backend



```
autoreserve@AutoreserveUbuntu:~$ ls -l ~/autoreserve-deploy/backend
total 64856
-rw-rw-r-- 1 autoreserve autoreserve 66410898 Jun 14 22:48 autoreserve-backend-0.0.1-SNAPSHOT.jar
autoreserve@AutoreserveUbuntu:~$
```

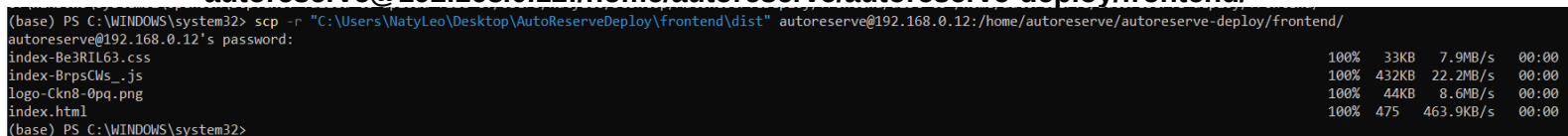
La preparación del entorno y la transferencia segura de los componentes constituyen una etapa fundamental dentro del proceso de implantación de aplicaciones empresariales, permitiendo garantizar que la infraestructura se encuentre debidamente configurada antes de iniciar la construcción y ejecución de los contenedores que conformarán la solución AutoReserve.

De manera similar, se procedió a transferir al servidor Ubuntu el artefacto correspondiente al frontend de AutoReserve, generado previamente mediante el proceso de construcción del proyecto con Vite. Para ello, se utilizó nuevamente el protocolo SCP, garantizando la transferencia segura de los archivos estáticos que conforman la interfaz web de la aplicación.

Figura 27. Transferencia del directorio compilado del frontend mediante SCP.

scp -r "C:\Users\NatyLeo\Desktop\AutoReserveDeploy\frontend\dist"

autoreserve@192.168.0.12:/home/autoreserve/autoreserve-deploy/frontend/



```
(base) PS C:\WINDOWS\system32> scp -r "C:\Users\NatyLeo\Desktop\AutoReserveDeploy\frontend\dist" autoreserve@192.168.0.12:/home/autoreserve/autoreserve-deploy/frontend/
autoreserve@192.168.0.12's password:
index-Be3RIL63.css                                                    100% 33KB 7.9MB/s 00:00
index-BrpsCws_.js                                                    100% 432KB 22.2MB/s 00:00
logo-Ckn8-0pq.png                                                    100% 44KB 8.6MB/s 00:00
index.html                                                            100% 475 463.9KB/s 00:00
(base) PS C:\WINDOWS\system32>
```

Una vez completada la transferencia, se verificó la presencia del directorio correspondiente dentro de la estructura de despliegue del servidor, confirmando que tanto el backend como

el frontend del sistema AutoReserve se encontraban disponibles para iniciar la fase de construcción y despliegue mediante contenedores Docker.

Figura 28. Verificación del directorio del frontend transferido al servidor Ubuntu.ls -l

~/autoreserve-deploy/frontend/dist

```
autoreserve@AutoreserveUbuntu:~$ ls -l ~/autoreserve-deploy/frontend/dist
total 12
drwx----- 2 autoreserve autoreserve 4096 Jun 14 23:20 assets
drwx----- 2 autoreserve autoreserve 4096 Jun 14 23:20 images
-rw-rw-r-- 1 autoreserve autoreserve  475 Jun 14 23:20 index.html
autoreserve@AutoreserveUbuntu:~$
```

10. Construcción y despliegue mediante Docker

Con el entorno de despliegue preparado y los artefactos del sistema disponibles en el servidor Ubuntu, se procedió a implementar una estrategia de contenerización basada en Docker. Esta aproximación permite encapsular los componentes del sistema junto con sus dependencias, garantizando la portabilidad, reproducibilidad y facilidad de despliegue en diferentes infraestructuras.

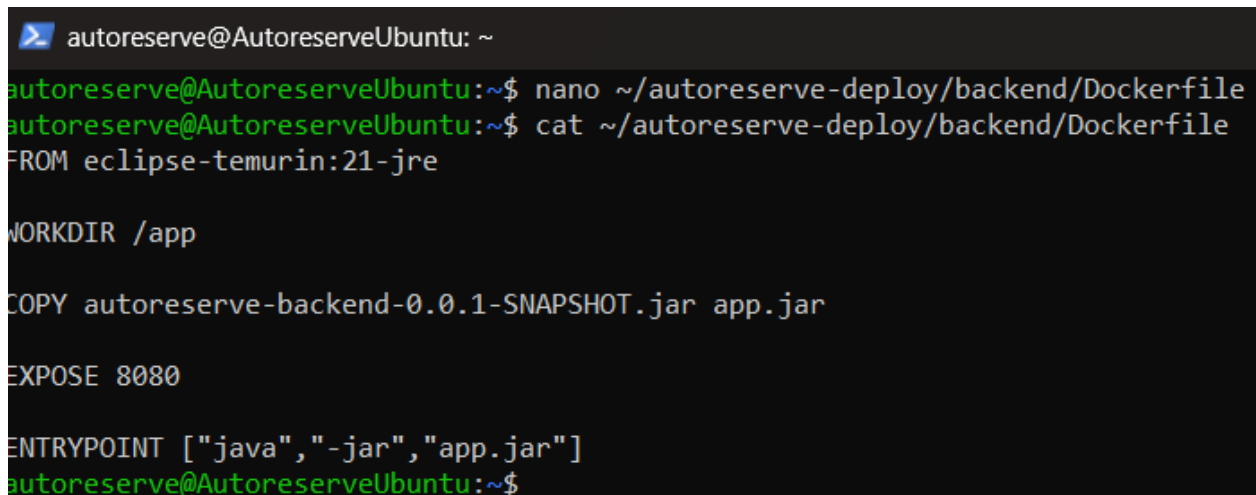
Como primera etapa, se construyó el Dockerfile correspondiente al backend de AutoReserve. Este archivo define la imagen base utilizada, el directorio de trabajo, la incorporación del artefacto ejecutable generado durante el proceso de construcción del proyecto y el comando responsable de iniciar la aplicación Spring Boot.

La imagen base seleccionada fue Eclipse Temurin con Java Runtime Environment versión 21, debido a su compatibilidad con la versión de Java empleada durante el desarrollo del sistema. Asimismo, se configuró la exposición del puerto 8080, utilizado por el backend para atender las solicitudes provenientes del frontend.

Figura 29. Creación del Dockerfile del backend de AutoReserve.

```
nano ~/autoreserve-deploy/backend/Dockerfile
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY autoreserve-backend-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","app.jar"]

cat ~/autoreserve-deploy/backend/Dockerfile
```



```
autoreserve@AutoreserveUbuntu: ~
autoreserve@AutoreserveUbuntu:~$ nano ~/autoreserve-deploy/backend/Dockerfile
autoreserve@AutoreserveUbuntu:~$ cat ~/autoreserve-deploy/backend/Dockerfile
FROM eclipse-temurin:21-jre

WORKDIR /app

COPY autoreserve-backend-0.0.1-SNAPSHOT.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java","-jar","app.jar"]
autoreserve@AutoreserveUbuntu:~$
```

Una vez definido el Dockerfile del backend, se procedió a construir la imagen Docker correspondiente utilizando el motor de contenedores instalado en Ubuntu Server. Durante este proceso, Docker descargó la imagen base requerida y generó una nueva imagen que encapsula el backend de AutoReserve junto con todas las dependencias necesarias para su ejecución.

La imagen resultante fue etiquetada como "autoreserve-backend:1.0", facilitando su identificación y posterior utilización durante la orquestación de los servicios del sistema. Posteriormente, se verificó su correcta creación mediante la consulta del repositorio local de imágenes Docker disponible en el servidor.

Figura 30. Construcción de la imagen Docker del backend de AutoReserve.
sudo docker build -t autoreserve-backend:1.0 .

```
autoreserve@AutoreserveUbuntu:~$ ls /media
autoreserve@AutoreserveUbuntu:~$ ls /media/AutoReserveDeploy
ls: cannot access '/media/AutoReserveDeploy': No such file or directory
autoreserve@AutoreserveUbuntu:~$ cd ~/autoreserve-deploy/backend
autoreserve@AutoreserveUbuntu:~/autoreserve-deploy/backend$ sudo docker build -t autoreserve-backend:1.0 .
[sudo: authenticate] Password:
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
              Install the buildx component to build images with BuildKit:
              https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 66.41MB
Step 1/5 : FROM eclipse-temurin:21-jre
21-jre: Pulling from library/eclipse-temurin
74e368405305: Pulling fs layer
d8d245b274e0: Pulling fs layer
a1ec5cc057cf: Pulling fs layer
1c24395ddd46: Pulling fs layer
993d8fd138c9: Pulling fs layer
6f5c5aa4e145: Pulling fs layer
a1ec5cc057cf: Download complete
74e368405305: Download complete
1c24395ddd46: Download complete
6e88a47573a8: Download complete
74e368405305: Pull complete
d8d245b274e0: Pull complete
Digest: sha256:194b590649efee5555de080c634905a30c2885190210e37ada26370ed2a9c781
Status: Downloaded newer image for eclipse-temurin:21-jre
---> 194b590649ef
Step 2/5 : WORKDIR /app
---> Running in 87604c24d3e6
---> Removed intermediate container 87604c24d3e6
---> 025efb9c01ac
Step 3/5 : COPY autoreserve-backend-0.0.1-SNAPSHOT.jar app.jar
---> 536125987079
Step 4/5 : EXPOSE 8080
---> Running in cbdc33c818ea
---> Removed intermediate container cbdc33c818ea
---> a77b983ebd03
Step 5/5 : ENTRYPOINT ["java","-jar","app.jar"]
---> Running in 43148c899bba
---> Removed intermediate container 43148c899bba
---> afcafb1653f4
--> Successfully built afcafb1653f4
--> Successfully tagged autoreserve-backend:1.0
autoreserve@AutoreserveUbuntu:~/autoreserve-deploy/backend$ _
```

Figura 31. Verificación de la imagen Docker del backend creada exitosamente.

sudo docker images

```
autoreserve@AutoreserveUbuntu:~/autoreserve-deploy/backend$ sudo docker images
[sudo: authenticate] Password:
IMAGE                ID                DISK USAGE  CONTENT SIZE  EXTRA
autoreserve-backend:1.0  afcafb1653f4      580MB       175MB
eclipse-temurin:21-jre  194b590649ef      459MB       120MB
autoreserve@AutoreserveUbuntu:~/autoreserve-deploy/backend$
```

Es importante destacar que, además de la imagen personalizada del backend, se observó la presencia de la imagen base Eclipse Temurin 21 JRE, utilizada por Docker durante el proceso de construcción. Esto evidencia la reutilización de imágenes oficiales como fundamento para la creación de contenedores específicos de la aplicación.

Posteriormente, se construyó el Dockerfile correspondiente al frontend de AutoReserve. Debido a que la aplicación frontend fue previamente compilada mediante Vite, únicamente fue necesario copiar los archivos estáticos generados hacia un servidor web ligero basado en Nginx.

Se seleccionó la imagen oficial Nginx Alpine debido a su reducido tamaño y eficiencia para la publicación de contenido estático. Asimismo, se configuró la exposición del puerto 80, utilizado convencionalmente para la prestación de servicios web.

Figura 32. Creación del Dockerfile del frontend de AutoReserve.

nano ~/autoreserve-deploy/frontend/Dockerfile

FROM nginx:alpine

COPY dist/ /usr/share/nginx/html

EXPOSE 80

cat ~/autoreserve-deploy/frontend/Dockerfile

```
autoreserve@AutoreserveUbuntu:~$ nano ~/autoreserve-deploy/frontend/Dockerfile
autoreserve@AutoreserveUbuntu:~$ cat ~/autoreserve-deploy/frontend/Dockerfile
FROM nginx:alpine

COPY dist/ /usr/share/nginx/html

EXPOSE 80
autoreserve@AutoreserveUbuntu:~$
```


Una vez definido el Dockerfile del frontend, se procedió a construir la imagen Docker correspondiente. Durante este proceso, Docker descargó la imagen oficial de Nginx y generó una nueva imagen personalizada que incorpora los recursos estáticos del sistema AutoReserve.

Posteriormente, se verificó la existencia de las imágenes construidas para el frontend y el backend, confirmando que ambos componentes del aplicativo se encontraban listos para su integración y despliegue conjunto mediante Docker Compose.

Figura 33. Construcción de la imagen Docker del frontend de AutoReserve.

sudo docker build -t autoreserve-frontend:1.0 .

```
autoreserve@AutoreserveUbuntu:~/autoreserve-deploy/backend$ cd ~/autoreserve-deploy/frontend
autoreserve@AutoreserveUbuntu:~/autoreserve-deploy/frontend$ sudo docker build -t autoreserve-frontend:1.0 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
              Install the buildx component to build images with BuildKit:
              https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  527.4kB
Step 1/3 : FROM nginx:alpine
alpine: Pulling from library/nginx
fbaed3f7fcbe: Pulling fs layer
abaae85d1626: Pulling fs layer
94d083cf706a: Pulling fs layer
43f834d60d8a: Pulling fs layer
e654dbbbb9e1: Pulling fs layer
de1b677d8c00: Pulling fs layer
6a0ac1617861: Pulling fs layer
a5008f4a4b25: Pulling fs layer
a5008f4a4b25: Download complete
e654dbbbb9e1: Download complete
94d083cf706a: Download complete
43f834d60d8a: Download complete
-
abaae85d1626: Pull complete
a5008f4a4b25: Pull complete
e654dbbbb9e1: Pull complete
94d083cf706a: Pull complete
fbaed3f7fcbe: Download complete
fbaed3f7fcbe: Pull complete
Digest: sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816598e27fb910ed2e621e589a
Status: Downloaded newer image for nginx:alpine
--> 8b1e78743a03
Step 2/3 : COPY dist/ /usr/share/nginx/html
--> bcf6ceeb8507
Step 3/3 : EXPOSE 80
--> Running in b8f90b10a1cc
--> Removed intermediate container b8f90b10a1cc
--> b2e87f1ce59a
--> Successfully built b2e87f1ce59a
--> Successfully tagged autoreserve-frontend:1.0
autoreserve@AutoreserveUbuntu:~/autoreserve-deploy/frontend$
```

Figura 34. Verificación de las imágenes Docker del sistema AutoReserve.

sudo docker images

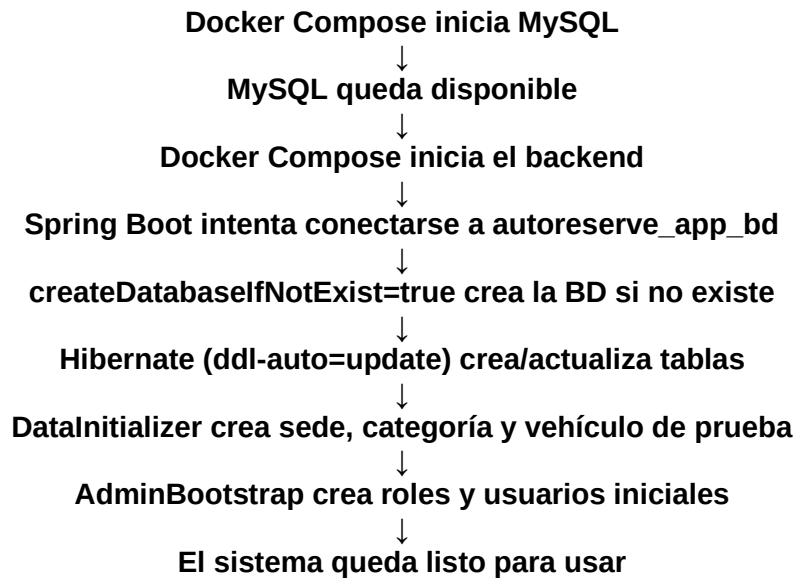
```
autoreserve@AutoreserveUbuntu:~/autoreserve-deploy/frontend$ sudo docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
autoreserve-backend:1.0	afcafb1653f4	580MB	175MB	
autoreserve-frontend:1.0	b2e87f1ce59a	93.4MB	26.2MB	
eclipse-temurin:21-jre	194b590649ef	459MB	120MB	
nginx:alpine	8b1e78743a03	93.6MB	27MB	

```
autoreserve@AutoreserveUbuntu:~/autoreserve-deploy/frontend$
```

Ahora vamos con el siguiente paso práctico: crear el **docker-compose.yml**

El flujo será este:



Como siguiente etapa del despliegue, se implementó un mecanismo de orquestación basado en Docker Compose, permitiendo administrar de manera centralizada los diferentes servicios que conforman la solución AutoReserve.

Para ello, se definió un archivo denominado `docker-compose.yml`, responsable de coordinar la ejecución de los contenedores correspondientes a la base de datos MySQL, el backend desarrollado con Spring Boot y el frontend desplegado mediante Nginx.

La configuración establecida define aspectos fundamentales como los puertos de exposición, nombres de contenedores, dependencias entre servicios, políticas de reinicio y

persistencia de datos mediante volúmenes, facilitando así la reproducción del entorno completo de ejecución en cualquier infraestructura compatible con Docker.

Figura 35. Creación y verificación del archivo docker-compose.yml para la orquestación de AutoReserve.

```
nano docker-compose.yml
```

```
services:
```

```
  mysql:
```

```
    image: mysql:8.0
```

```
    container_name: autoreserve-mysql
```

```
    restart: always
```

```
    environment:
```

```
      MYSQL_ROOT_PASSWORD: 1234
```

```
    ports:
```

```
      - "3306:3306"
```

```
    volumes:
```

```
      - ./mysql:/var/lib/mysql
```

```
  backend:
```

```
    image: autoreserve-backend:1.0
```

```
    container_name: autoreserve-backend
```

```
    restart: always
```

```
    depends_on:
```

```
      - mysql
```

```
    ports:
```

```
      - "8080:8080"
```

```
  frontend:
```

```
    image: autoreserve-frontend:1.0
```

container_name: autoreserve-frontend

restart: always

depends_on:

- backend

ports:

- "80:80"

cat docker-compose.yml

```
autoreserve@AutoreserveUbuntu: ~/autoreserve-deploy
autoreserve@AutoreserveUbuntu:~/autoreserve-deploy$ nano docker-compose.yml
autoreserve@AutoreserveUbuntu:~/autoreserve-deploy$ cat docker-compose.yml
services:
  mysql:
    image: mysql:8.0
    container_name: autoreserve-mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: 1234
    ports:
      - "3306:3306"
    volumes:
      - ./mysql:/var/lib/mysql

  backend:
    image: autoreserve-backend:1.0
    container_name: autoreserve-backend
    restart: always
    depends_on:
      - mysql
    ports:
      - "8080:8080"

  frontend:
    image: autoreserve-frontend:1.0
    container_name: autoreserve-frontend
    restart: always
    depends_on:
      - backend
    ports:
      - "80:80"
autoreserve@AutoreserveUbuntu:~/autoreserve-deploy$
```

Despliegue automatizado de AutoReserve mediante Docker Compose

Una vez definidas las imágenes Docker del frontend y del backend, así como la configuración de la base de datos MySQL, se procedió a ejecutar el despliegue automatizado del sistema utilizando Docker Compose.

Para ello, desde la carpeta principal del proyecto se ejecutó el siguiente comando:

sudo docker compose up -d

Este comando permitió la creación automática de la red privada del proyecto y el inicio coordinado de los servicios definidos en el archivo docker-compose.yml. Como resultado, se levantaron satisfactoriamente tres contenedores independientes correspondientes a la base de datos MySQL, al backend desarrollado en Spring Boot y al frontend desplegado mediante Nginx.

Figura 37. Ejecución del comando docker compose up -d para el despliegue automatizado de AutoReserve.



```
autoreserve@AutoReserve:~/autoreserve-deploy$ sudo docker compose up -d
[+] Running 14/14
✔ mysql Pulled
✔ 96d30d9fbee8 Pull complete
✔ ecf85873f64e Pull complete
✔ 4c8a3e0d4e4b Pull complete
✔ a63160a5eda1 Pull complete
✔ 7334d1db9f8d Pull complete
✔ 49ec2dab01d9 Pull complete
✔ ab24264a27e9 Pull complete
✔ e3e5d1ac74c1 Pull complete
✔ 6ef6c7b50a93 Pull complete
✔ 0d74d266095b Pull complete
✔ 297084cfe470 Pull complete
✔ 796812c73292 Download complete
✔ af166387641d Download complete
[+] Running 4/4
✔ Network autoreserve-deploy_default Created
✔ Container autoreserve-mysql Started
✔ Container autoreserve-backend Started
✔ Container autoreserve-frontend Started
autoreserve@AutoReserve:~/autoreserve-deploy$
```

La figura muestra el proceso mediante el cual Docker Compose crea la red interna de comunicación y pone en funcionamiento los contenedores correspondientes a la base de datos MySQL, el backend desarrollado con Spring Boot y el frontend servido mediante Nginx.

Validación funcional del despliegue

Con el objetivo de verificar el correcto funcionamiento de los servicios desplegados, se realizaron pruebas básicas de conectividad utilizando el comando curl desde el sistema operativo Ubuntu Server.

En primer lugar, se validó el acceso al frontend desplegado mediante Nginx a través del puerto 80. La respuesta obtenida correspondió al documento HTML principal generado

durante el proceso de compilación de React y Vite, evidenciando que la interfaz gráfica de AutoReserve se encontraba correctamente publicada.

Posteriormente, se verificó la disponibilidad del backend desarrollado con Spring Boot mediante una solicitud HTTP dirigida al puerto 8080. El sistema respondió con un código de estado HTTP 401 (Unauthorized), comportamiento esperado debido a la configuración de seguridad implementada en la aplicación, confirmando así que el servidor de aplicaciones se encontraba operativo y procesando solicitudes correctamente.

Figura 38. Validación del frontend desplegado mediante Nginx utilizando el comando curl http://localhost y Validación del backend desplegado mediante Spring Boot utilizando el comando curl http://localhost:8080.

```
autoreserve@AutoreserveUbuntu:~/autoreserve-deploy$ curl http://localhost
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/assets/logo-Ckn8-0pq.png" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>auto-reserve</title>
    <script type="module" crossorigin src="/assets/index-BrpsCWS_.js"></script>
    <link rel="stylesheet" crossorigin href="/assets/index-Be3RIL63.css">
  </head>
  <body>
    <div id="root"></div>
  </body>
</html>
autoreserve@AutoreserveUbuntu:~/autoreserve-deploy$ curl http://localhost:8080
{"timestamp":"2026-06-15T01:52:20.862+00:00","status":401,"error":"Unauthorized","path":"/"}autoreserve@AutoreserveUbuntu:~/autoreserve-deploy$
```

Validación de la inicialización automática del sistema

Como etapa final del proceso de instalación, se realizó la revisión de los registros de ejecución (logs) del contenedor correspondiente al backend de AutoReserve mediante Docker.

El análisis de dichos registros permitió evidenciar el correcto arranque del servidor de aplicaciones Spring Boot, la conexión exitosa con la base de datos MySQL y la ejecución satisfactoria de los mecanismos automáticos de inicialización implementados en el sistema.

Durante esta fase se verificó la creación automática de los datos maestros mínimos requeridos para la operación del aplicativo, incluyendo la sede principal, la categoría inicial de vehículos, el vehículo de prueba y los usuarios predeterminados correspondientes a los roles administrativos y de cliente.

La evidencia obtenida demuestra que el proceso de despliegue no requiere intervenciones manuales sobre la base de datos, garantizando una instalación reproducible, consistente y preparada para su utilización inmediata en el entorno del cliente.

Figura 39. Validación de la inicialización automática del backend mediante revisión de logs del contenedor Docker.

sudo docker logs autoreserve-backend | tail -50

```
>> Branch y Category creados (si no existian).
>> Auto de prueba creado correctamente.
>> Usuario admin creado (si no existia).
>> Usuario cliente creado (si no existia).
2026-06-15T01:52:20.823Z INFO 1 --- [autoreserve-backend] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
--- DEBUG: 401 Unauthorized en: /
autoreserve@autoreserveubuntu:~/autoreserve-deploy$
```

11. Resultados obtenidos

Como resultado del presente ejercicio, se logró desplegar exitosamente el producto mínimo viable AutoReserve sobre una infraestructura virtualizada basada en Ubuntu Server ejecutado en Oracle VirtualBox.

La solución fue implementada mediante una arquitectura compuesta por contenedores Docker independientes para la base de datos MySQL, el backend desarrollado con Java Spring Boot y el frontend construido con React y Vite, administrados de forma centralizada mediante Docker Compose.

Asimismo, se comprobó el correcto funcionamiento de los servicios desplegados mediante pruebas de conectividad y validación funcional, verificando la disponibilidad del frontend, la operación del backend y la inicialización automática de la información básica requerida por el sistema.

Los resultados obtenidos evidencian la viabilidad técnica del despliegue propuesto y la capacidad del aplicativo para ser instalado de manera controlada y repetible en entornos similares a los que podrían encontrarse en organizaciones del sector de renta de vehículos.

12. Conclusiones

Primera. El uso de tecnologías de virtualización y contenedores permitió simular de manera realista el proceso de instalación de una solución web empresarial, facilitando la validación previa del procedimiento antes de su implementación en ambientes productivos.

Segunda. La utilización de Docker y Docker Compose simplificó significativamente el despliegue de AutoReserve, permitiendo automatizar la configuración de la infraestructura y reduciendo la posibilidad de errores asociados a instalaciones manuales.

Tercera. La implementación de perfiles de configuración en Spring Boot para los entornos de desarrollo, pruebas y producción contribuyó a mejorar la mantenibilidad, portabilidad y calidad del software, alineando el proyecto con prácticas utilizadas en escenarios profesionales.

Cuarta. La inicialización automática de la base de datos y de los datos mínimos requeridos para el funcionamiento del sistema permitió obtener una instalación completamente funcional sin necesidad de ejecutar scripts manuales, favoreciendo la experiencia de implementación para futuros clientes.

Quinta. El desarrollo de esta actividad fortaleció las competencias relacionadas con despliegue de aplicaciones, administración básica de servidores Linux, uso de herramientas de virtualización y gestión de contenedores, constituyendo una experiencia práctica relevante para el ejercicio profesional en el área del desarrollo de software.

13. Referencias

- Docker Inc. (2025). Docker Documentation. <https://docs.docker.com/>
- Nginx, Inc. (2025). NGINX Documentation. <https://nginx.org/en/docs/>
- Oracle Corporation. (2025). Oracle VM VirtualBox User Manual. <https://www.virtualbox.org/manual/>
- Oracle Corporation. (2025). MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/>
- Spring. (2025). Spring Boot Reference Documentation. <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- Ubuntu. (2025). Ubuntu Server Documentation. <https://ubuntu.com/server/docs>

- VMware Tanzu. (2025). Spring Framework Documentation.
<https://docs.spring.io/spring-framework/reference/>